

● 模型接入渠道实测报告

Kimi-K3 双通道实测：we2ai 中转 vs 官方 API 直连

同一套功能用例与压测脚本，分别打向 we2ai 中转网关与 Moonshot 官方 API，用于评估两条接入路径在协议兼容性、行为一致性与响应延迟上的真实差异，供渠道选型参考。

模型 kimi-k3 测试日期 2026-08-24 用例规模 28 项 (19 基础 + 9 附加) 压测参数 3 会话 · seed 1 · 120s 超时



两条路径打的是完全相同的用例与请求体：**we2ai 路径**在客户端与模型之间多一层中转，请求/响应有被网关改写的可能；**官方路径**没有中间层，是 Moonshot 原生行为。下文的协议字段差异（如 usage、JSON 围栏）与官方侧的过载表现，都要结合这个结构差异来看。

● 要点摘要

两条通道都没有达到验收标准，但卡住的地方不一样

下面是完整测试的浓缩结论；决策所需的关键数字都在两张卡片里，细节展开见后续章节。

● we2ai 中转 api.we2ai.com 15/19 基础用例通过 3.3s 端到端延迟 P50 0 压测失败请求 总体判定 · FAIL	● 官方 API 直连 api.moonshot.cn 13/19 基础用例通过 7.2s 端到端延迟 P50 1 压测失败请求 总体判定 · FAIL
--	---

三句话讲完

- 两渠道请求体、能力声明完全一致，但 6 项用例的通过/失败结果不同，说明 kimi-k3 在两个渠道上的行为一致性并不好，不能假设"哪边测过另一边就一定一样"。
- we2ai 中转的首字与端到端延迟中位数明显更低（约为官方的一半），但尾部延迟（P99）更长、抖动更大；官方直连本次测试窗口内多次触发 429 engine_overloaded_error，压测阶段也出现 1 次真实过载失败。
- 结构化 JSON 输出在两个渠道都可能被套上 Markdown 代码块围栏，谁更"干净"并不固定，建议应用层统一做兜底解析。

一 · 测试环境

两条通道除入口地址外，其余条件完全对齐

模型 ID	kimi-k3（两渠道一致）
API 端点	we2ai：https://api.we2ai.com/v1/chat/completions · 官方：https://api.moonshot.cn/v1/chat/completions
测试日期	2026-08-24（两渠道同日、先后执行）
用例规模	28 项用例，19 项计入基础通过率分母，9 项为附加校准/边界用例不计入分母
压测参数	3 个并发会话，随机种子 1，单请求超时 120s，两渠道参数完全相同
一致性保证	两渠道使用完全相同的用例定义、能力声明与请求体，仅 base URL 与 API Key 不同，结果差异可直接归因于渠道本身

二 · 能力声明

本轮测试覆盖的能力范围

未声明的能力（如 image_url、tool_choice_function）会被测试框架自动跳过，不计入通过率。

工具调用 支持

tool_choice_function 未声明

思考开关 4 种方式声明

默认行为 默认开启思考

reasoning_effort 支持

图片/视频 仅 Base64，不含 URL 输入

三 · 功能测试结果

28 项用例逐一对照

紫色圆点标记的行是两渠道结果不同的用例，共 6 项；延迟条按同一行内的最大值归一化，仅用于比较该用例下两渠道谁更快。

● we2ai 延迟 ● 官方 API 延迟 ● 两渠道结果不同

用例	WE2AI	官方 API	平均延迟	说明
连通性				
基础连通性 connectivity.basic	● 通过	● 通过	5.5s 8.4s	
协议 / 流式				
非流式 usage 字段 protocol.usage_nonstream	● 通过	● 通过	3.1s 9.0s	
流式 usage 字段 protocol.usage_stream	● 通过	● 失败	4.3s 11.3s	● 渠道差异 · 见发现 1
流式响应完整性 protocol.stream_integrity	● 通过	● 通过	5.5s 12.6s	
多模态输入				
图片 URL 输入 multimodal.image_url	● 未声明	● 未声明	—	能力未声明，跳过
图片 Base64 输入 multimodal.image_base64	● 通过	● 通过	9.7s 4.7s	
视频 URL 输入 multimodal.video_url	● 未声明	● 未声明	—	能力未声明，跳过
视频 Base64 输入 multimodal.video_base64	● 通过	● 通过	4.8s 4.0s	
工具调用				
工具调用能力 tool.call_capability	● 通过	● 通过	4.3s 4.4s	
tool_choice 默认值 tool.choice_default	● 通过	● 通过	4.7s 6.3s	
tool_choice = auto tool.choice_auto	● 通过	● 通过	5.2s 5.1s	
tool_choice = required tool.choice_required	● 通过	● 通过	4.9s 3.8s	
tool_choice = none tool.choice_none	● 通过	● 通过	7.1s 3.8s	
tool_choice = 指定函数 tool.choice_function	● 未声明	● 未声明	—	能力未声明，跳过
tool_choice = allowed_tools tool.choice_allowed_tools	● 通过	● 通过	7.1s 4.0s	
思考控制				
enable_thinking 开关 thinking.enable_thinking	● 失败	● 失败	4.6s 7.2s	两渠道一致 · 见发现 2

用例	WE2AI	官方 API	平均延迟	说明
thinking_type 开关 thinking.thinking_type	● 通过	● 失败	3.4s 11.5s	● 渠道差异 · 见发现 3
chat_template_kwargs.enable_thinking thinking.chat_template_kwargs_enable_thinking	● 失败	● 失败	9.0s 10.2s	两渠道一致 · 见发现 3
chat_template_kwargs.thinking thinking.chat_template_kwargs_thinking	● 失败	● 失败	11.0s 8.0s	两渠道一致 · 见发现 2
默认思考行为 thinking.default_behavior	● 通过	● 失败	6.8s 9.7s	● 渠道差异 · 见发现 3
结构化输出				
JSON Schema 严格模式 structured.json_schema	● 通过	● 通过	4.5s 7.4s	
JSON Object 模式 structured.json_object	● 失败	● 通过	10.0s 7.8s	● 渠道差异 · 见发现 4
附加用例（不计入基础分母）				
reasoning_effort 分档采样 reasoning_effort.scaling	● 失败	● 失败	25.4s 44.2s	两渠道一致 · 见发现 5
max_tokens 负值校验 input_validation.max_tokens_negative	● 通过	● 通过	0.3s 0.1s	
tool 结果内容对象校验 input_validation.tool_result_content_object	● 通过	● 通过	1.0s 0.1s	
重复 content parts 校验 input_validation.duplicate_content_parts	● 失败	● 失败	10.2s 10.4s	两渠道一致 · 见发现 6
非法 role 校验 input_validation.invalid_role	● 失败	● 通过	16.9s 0.1s	● 渠道差异 · 见发现 6
损坏的图片 Base64 校验 input_validation.corrupted_image_base64	● 通过	● 通过	13.9s 0.1s	

● 四 · 关键发现

6 处真正值得关注的细节

编号与上方结果表的「说明」列一一对应，可通过表格直接跳转到对应发现。

01 官方直连流式响应缺少 usage 字段，we2ai 正常

渠道差异

两渠道请求体完全一致（stream:true，未显式传 stream_options.include_usage）。we2ai 返回的流式响应末尾带有 usage 字段，官方直连的流里没有这个字段。属于协议层的真实差异——we2ai 中转很可能默认补全了这个参数，官方原生行为是不传就不返回。

02 两种"关闭思考"参数在两渠道都不生效

两渠道一致

对 enable_thinking=false 与 chat_template_kwargs.thinking=false 两种关闭方式，两个渠道的模型都仍然返回了思考内容（reasoning_content）。两渠道现象一致，说明是 kimi-k3 模型本身对这两种取值不敏感，与走哪个渠道无关。

03 官方直连在思考类用例上多次触发过载

渠道差异

thinking_type、chat_template_kwargs.enable_thinking、default_behavior 三个用例，官方直连均返回 HTTP 429：

```
{"error":{"message":"The engine is currently overloaded, please try again later"},"type":"engine_overloaded_error"}
```

we2ai 中转对同样的请求全部正常返回。压测阶段官方也出现 1 次同类型失败，we2ai 压测零失败。本次测试窗口内官方 API 出现多次过载，无法仅凭一轮测试判断是系统性容量差异还是偶发拥堵，建议择时重跑交叉验证。

04 JSON 结构化输出的围栏包裹方向不固定

渠道差异

两渠道请求体逐字节相同（response_format:{type:"json_object"}）。we2ai 返回的内容被 Markdown 代码块围栏包裹，解析为非法 JSON 而失败；官方直连返回的是干净 JSON。这不代表官方"更规范"——依赖严格 JSON 输出的场景，无论走哪个渠道都建议在应用层做一层围栏剥离的兜底解析。

05 reasoning_effort 分档采样两渠道都不稳定，但原因不同

两渠道一致

we2ai 在 max 档的一次采样返回 HTTP 503，导致该档采样不完整；官方全部采样请求都成功，但 low 档 3 次采样结果分散，没有形成多数结果。一个是传输层偶发失败，一个是模型输出本身在低强度档位下方差较大，都不是渠道路由逻辑导致的。

06 输入校验：重复内容两渠道都不拦，非法角色只有官方拦

渠道差异

对重复 content parts 的非法输入，两渠道都返回 HTTP 200 直接当合法请求处理，现象一致。但对非法 role 字段，we2ai 同样直接放行，官方直连正确拒绝了该请求。这是本次唯一体现"官方直连输入校验更严格"的用例。

● 五 · 压测对比

3 会话压测：延迟与吞吐

参数：-total-sessions 3 -seed 1 -request-timeout 120s，两渠道完全相同。样本量较小，吞吐数字仅供参考，不建议单独作为容量结论。

总请求数

91 88

we2ai 官方 API

官方含 1 次 429 失败

总耗时

814.5 902.3

we2ai 官方 API

吞吐 REQ/S

0.112 0.096

we2ai 官方 API

两渠道均低于基线，判定 FAIL

吞吐 INPUT TOK/S

2275.97 1963.52

we2ai 官方 API

吞吐 OUTPUT TOK/S

4.39 5.47

we2ai 官方 API

官方单请求输出吞吐略高

失败请求

0 1

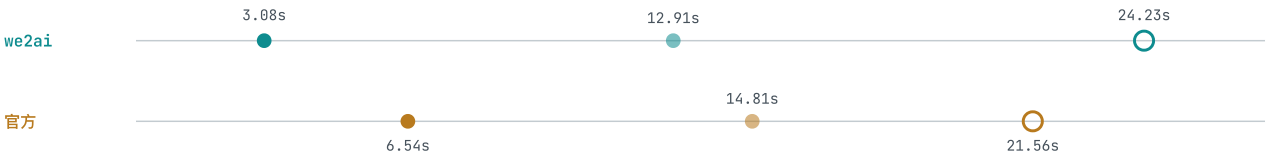
we2ai 官方 API

官方: HTTP 429

● we2ai ● 官方 API ● P50 ● P90 ○ P99

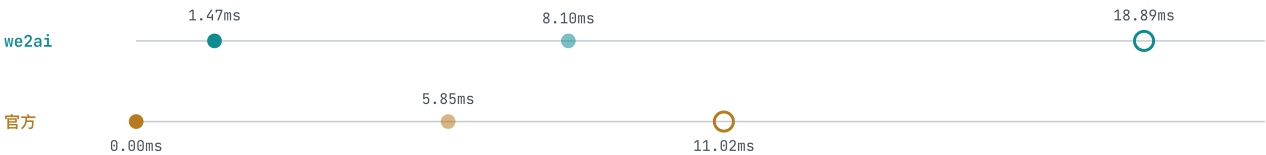
TTFT · 首字延迟

验收判定仅参考 P50，其余分位仅记录



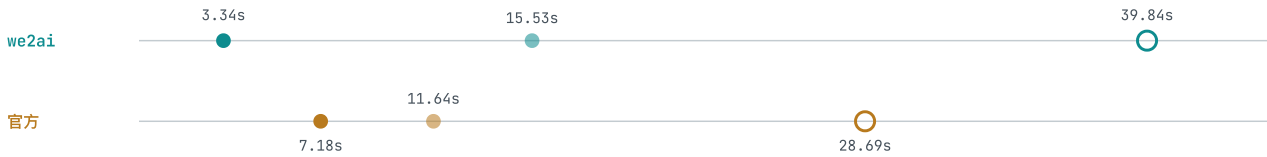
TPOT · 逐 token 耗时

验收判定仅参考 P50，其余分位仅记录



端到端延迟

验收判定仅参考 P50，其余分位仅记录



六 · 结论

怎么看这份结果

验收结论

两渠道 19 项基础用例通过率均未达到 100%（we2ai 15/19，官方 13/19），按验收标准均判定为 **整体 FAIL**。7 项失败用例中有 6 项两渠道结果不同，说明失败原因并不共通，需要分别处理。

真实渠道差异

- 流式响应 usage 字段：we2ai 有，官方无
- JSON 结构化输出围栏包裹：方向不固定，两渠道都可能出现
- 非法 role 校验：官方拒绝，we2ai 直接放行
- 官方直连在思考类用例上多次 429 过载，we2ai 未复现

模型本身的问题（与渠道无关）

- 两种"关闭思考"参数在两个渠道都不生效
- reasoning_effort 低档位输出方差偏大，采样不稳定
- 重复 content parts 的非法输入，两渠道都未做校验

压测层面

we2ai 中位数延迟明显更低，但 P99 尾部更长、抖动更大；官方直连本次出现 1 次真实 429 失败，与功能测试阶段的过载现象相互印证。吞吐两渠道均未达基线，建议后续加大会话数重跑以提升置信度。

给业务侧的建议

- 对首字延迟敏感的交互式场景，we2ai 中转当前实测更快；对尾部延迟稳定性要求高的场景，需留意 we2ai 的 P99 抖动更大。
- 官方直连的输入校验更严格，但本次测试窗口出现多次过载，建议错峰重跑一次交叉验证，排除偶发拥堵的可能。
- 依赖结构化 JSON 输出的场景，无论选哪个渠道，都建议在应用层加一层"剥离 Markdown 围栏"的兜底解析。

- 两渠道均未达到 100% 用例通过率，正式上线前建议与我们一起复核失败用例对具体业务场景的实际影响，而非直接以本报告作为唯一决策依据。